SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Scenario-Based Requirement Analysis
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	V. Karthik Harish	**Reg. No.:**	192511328
Date:	23/10/26	**Duration:**	60 minutes

Code of Conduct

*I **V. Karthik Harish (192511328)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student:

Annexure A – Scenario-Based Requirement Analysis

Instructions to Students:

Each student will be assigned an individual software project problem statement. Analyze the given scenario and prepare a complete Software Requirement Analysis document by following the steps below.

Question

Based on the **assigned problem statement**, perform a **Scenario-Based Requirement Analysis** and prepare a Software Requirement Specification (SRS) document. Your analysis should include the following:

1. **Study and Analyze the Scenario**
   - Carefully understand the given problem statement.
   - Identify the business domain, stakeholders, existing challenges, and system objectives.
2. **Identify Functional and Non-Functional Requirements**
   - List all functional requirements required by the proposed system.
   - Identify suitable non-functional requirements such as performance, security, reliability, usability, scalability, maintainability, and availability.
3. **Identify Actors and Use Cases**
   - Identify all primary and secondary actors interacting with the system.
   - List the major use cases associated with each actor.
   - Draw a Use Case Diagram representing the interactions between actors and the system.

4. **Prepare the Software Requirement Specification (SRS)**
   Develop an SRS document containing:
   - Introduction
   - Problem Description
   - Scope of the System
   - Functional Requirements
   - Non-Functional Requirements
   - Use Case Descriptions
   - Data Requirements
   - External Interface Requirements
   - System Features
5. **Identify Assumptions and Constraints**
   - List the assumptions considered while developing the system.
   - Identify technical, operational, economic, legal, and time-related constraints affecting the project.
6. **Validate Requirement Completeness and Consistency**
   - Verify whether all stakeholder requirements have been addressed.
   - Check for ambiguity, redundancy, inconsistency, and missing requirements.
   - Suggest improvements to enhance the quality and completeness of the requirement specification.

CO2

ASSESSMENT 1

Name: V Karthik Harish

Reg No: 192511328

Course: CSA1023 SOFTWARE ENGINEERING

Slot: B

Faculty: Dr. Karpagam V

Problem Statement: 39. Secure Digital Wallet and Payment Orchestration Platform

Table of Contents

Section	Title & Primary Engineering Deliverables	Focus Area
1.0	Introduction & Document Conventions	Purpose, Conventions, Intended Audience, Acronyms
2.0	Problem Description & Domain Analysis	Industry Context, Statement, Technical Pain Points (Fig 2.1)
3.0	Scope of the System & Boundary Analysis	In-Scope (Fig 3.1), Out-of-Scope, System Boundary (Fig 3.2)
4.0	Stakeholder Analysis & Influence Mapping	Identification, Power-Interest Grid (Fig 4.1), Communication
5.0	Actor Identification & Use Case Modeling	Primary/Secondary Actors, Use Case Model (Fig 5.1), Summary
6.0	Functional Requirements Specification	FR-01 to FR-32, State Machine (Fig 6.1), Reversal Logic
7.0	Non-Functional Requirements Specification	NFR-01 to NFR-29, Performance, Security, Capacity (Fig 7.1)
8.0	Detailed Use Case Descriptions	UC-05 Payment Sequence (Fig 8.1), UC-04 Add Funds, UC-09 Refund
9.0	System Architecture & Block Diagrams	Microservices (Fig 9.1), Flowchart (Fig 9.2), SAGA (Fig 9.4)
10.0	Data Requirements & Schema Specifications	ER Diagram (Fig 10.1), Data Dictionaries, Volume & Retention
11.0	External Interface Requirements	UI/UX, Hardware (NFC/Biometric), REST APIs, Gateway Adapters
12.0	System Features & Priority Traceability	Feature Priority Matrix (Fig 12.1), Feature-Requirement Mapping

Section	Title & Primary Engineering Deliverables	Focus Area
13.0	Assumptions, Dependencies & Constraints	Technical, Operational, Legal, Constraint Model (Fig 13.1)
14.0	Requirement Verification & Validation	V&V Strategy, Validation Matrix, Rubric Coverage (Fig 14.1)
15.0	Appendices & Engineering Sign-Off	Glossary, Standards, APIs, Domain Class Diagram (Fig 15.1)

1. Introduction

1.1 Purpose

This Software Requirement Specification document defines the complete functional and non-functional requirements for a **Secure Digital Wallet and Payment Orchestration Platform**. The platform coordinates payment requests across multiple financial services, maintaining transaction consistency and providing a fully auditable record of every operation. This document serves as the authoritative reference for all stakeholders during design, development, testing, and deployment phases.

1.2 Document Conventions

- **FR-xxx**: Functional Requirement identifier
- **NFR-xxx**: Non-Functional Requirement identifier
- **UC-xxx**: Use Case identifier
- **Bold**: Key terms or definitions
- ***Italic***: External system or document references
- `**Monospace**`: API endpoints, code, or configuration values

Priority Levels:

- **P1 — Critical**: Must be implemented; system cannot function without it
- **P2 — High**: Core functionality; required for production launch
- **P3 — Medium**: Enhances user experience; planned for initial release
- **P4 — Low**: Desirable; scheduled for future iterations

1.3 Intended Audience

- **Software Developers**: Implementation reference for modules, APIs, and data models
- **System Architects**: Architecture decisions, integration patterns, and component design
- **QA & Test Engineers**: Deriving test cases from functional and non-functional specifications
- **Project Managers**: Scope, constraints, and milestone tracking
- **Business Stakeholders**: Validating that the platform meets business objectives
- **Security Auditors**: Reviewing compliance requirements and security controls
- **Database Administrators**: Understanding the data model, relationships, and storage requirements

1.4 Definitions, Acronyms, and Abbreviations

- **Digital Wallet**: A software-based system that stores payment credentials and enables electronic transactions
- **Payment Orchestration**: The process of routing, managing, and reconciling payment flows across multiple gateways
- **Idempotency**: A property ensuring that performing an operation multiple times yields the same result as performing it once
- **PCI-DSS**: Payment Card Industry Data Security Standard
- **mTLS**: Mutual Transport Layer Security — both client and server authenticate each other
- **HMAC**: Hash-based Message Authentication Code
- **2FA**: Two-Factor Authentication

- **SAGA**: A design pattern for managing distributed transactions through a series of compensating actions
- **CI/CD**: Continuous Integration / Continuous Deployment
- **MTTR**: Mean Time To Recovery
- **MTBF**: Mean Time Between Failures
- **TPS**: Transactions Per Second
- **KYC**: Know Your Customer
- **AML**: Anti-Money Laundering
- **OTP**: One-Time Password

2. Problem Description & Domain Analysis

2.1 Industry Context

Digital payment systems form the backbone of modern commerce. These systems interact with multiple financial institutions, card networks, mobile payment providers, and banking APIs simultaneously. Each of these external services operates independently with its own protocols, response times, and failure modes.

The key challenges facing digital payment platforms today include:

- **Transaction State Inconsistency:** When a payment involves multiple services (e.g., debit from wallet, credit to merchant via gateway), a failure at any point can leave the overall transaction in an undefined state — money debited but not credited, or duplicate charges.
- **Duplicate Processing:** Network timeouts and retry mechanisms can cause the same payment to be processed more than once, leading to financial discrepancies.
- **Security Vulnerabilities:** Payment data is a high-value target. Weak authentication, unencrypted channels, or improper token management create risk.
- **Audit Trail Gaps:** Without comprehensive logging, resolving disputes and meeting regulatory obligations becomes impossible.
- **Scalability Bottlenecks:** Payment volumes spike during festivals, sales events, and month-end salary periods. Systems that cannot scale horizontally fail under peak load.

2.2 Problem Statement

Design and develop a **Secure Digital Wallet and Payment Orchestration Platform** that coordinates payment requests across multiple services while maintaining transaction consistency and auditability. The platform must provide secure APIs, robust authentication, transaction state management, idempotency guarantees, monitoring, logging, automated testing, CI/CD pipelines, and recovery mechanisms.

2.3 Current Pain Points

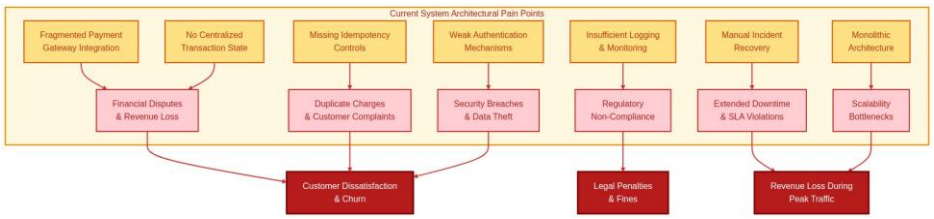


Figure 2.1: Current Industry Pain Points & Technical Bottlenecks in Legacy Payment Gateways

2.4 Proposed System Objectives

The platform addresses each identified pain point through specific technical solutions:

- **Fragmented gateway integration** (Unified Payment Orchestration Layer with adapter pattern): OBJ-01
- **No centralized transaction state** (Distributed state machine with SAGA pattern): OBJ-02
- **Missing idempotency** (Idempotency key enforcement at API gateway level): OBJ-03
- **Weak authentication** (OAuth 2.0 + 2FA + mTLS for service-to-service calls): OBJ-04
- **Insufficient logging** (Centralized structured logging with correlation IDs): OBJ-05
- **Manual incident recovery** (Automated health checks, circuit breakers, and rollback): OBJ-06
- **Monolithic architecture** (Microservices-based architecture with horizontal scaling): OBJ-07

3. Scope of the System

3.1 In-Scope Features

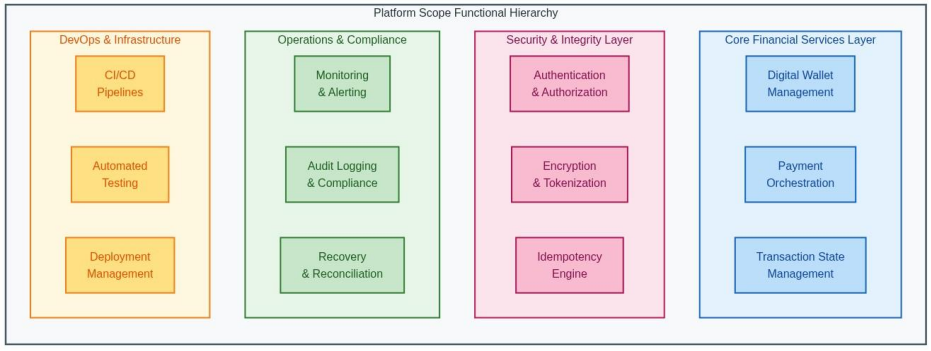


Figure 3.1: Complete System Scope & Functional Feature Decomposition Hierarchy

3.2 Out-of-Scope

The following are explicitly excluded from the current version:

- Cryptocurrency or blockchain-based payment processing
- Physical point-of-sale (POS) terminal integration
- Issuing of physical or virtual bank cards
- Credit lending or loan disbursement features
- Cross-border foreign exchange rate arbitrage
- Direct integration with central bank payment rails (e.g., UPI backend infrastructure)

3.3 System Boundary Diagram

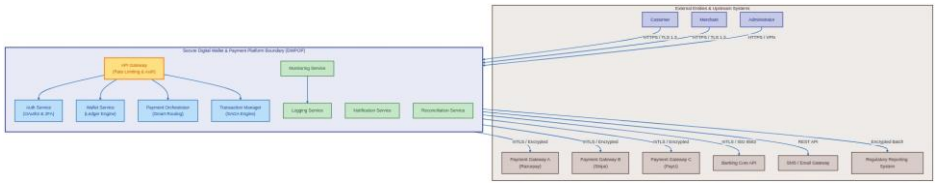


Figure 3.2: Enterprise System Context & External Boundary Interface Architecture

4. Stakeholder Analysis

4.1 Stakeholder Identification

Stakeholder	Role	Interest	Influence
End Customer	Initiates payments, manages wallet	Seamless and secure payment experience	High
Merchant	Receives payments, views settlements	Fast settlement, low failure rates	High
System Administrator	Manages platform, monitors health	System stability, quick incident response	High
Compliance Officer	Ensures regulatory adherence	Audit logs, PCI-DSS compliance	Medium
Payment Gateway Provider	Processes card/UPI/net-banking payments	Reliable API integration, SLA adherence	Medium
Banking Partner	Settles funds, manages accounts	Accurate reconciliation, fraud prevention	Medium
DevOps Engineer	Deploys, scales, and maintains services	CI/CD reliability, monitoring coverage	Medium
QA Engineer	Tests functionality and performance	Comprehensive test coverage, defect tracking	Low
Business Analyst	Defines requirements, validates features	Feature completeness, business alignment	Low

4.2 Stakeholder Influence–Interest Matrix



Figure 4.1: Stakeholder Influence vs. Interest Matrix (Power-Interest Grid)

5. Actor Identification & Use Case Modeling

5.1 Actor Classification

Primary Actors (Direct Users)

- **Customer** (Registers, manages wallet, initiates payments, views history): OAuth 2.0 + OTP-based 2FA
- **Merchant** (Views incoming payments, requests settlements, manages profile): OAuth 2.0 + API Key
- **System Administrator** (Configures system, manages users, monitors dashboards): OAuth 2.0 + Hardware Token 2FA

Secondary Actors (External Systems)

- **Payment Gateway** (Processes card, UPI, and net-banking transactions): REST API over mTLS
- **Banking System** (Handles fund settlements and account verification): Secure SFTP + API
- **SMS/Email Provider** (Sends OTPs, transaction alerts, and notifications): REST API with API Key
- **Fraud Detection Engine** (Screens transactions for suspicious patterns): gRPC with mutual auth
- **Regulatory Reporting System** (Receives compliance reports and audit data): Encrypted batch files

5.2 Use Case Diagram

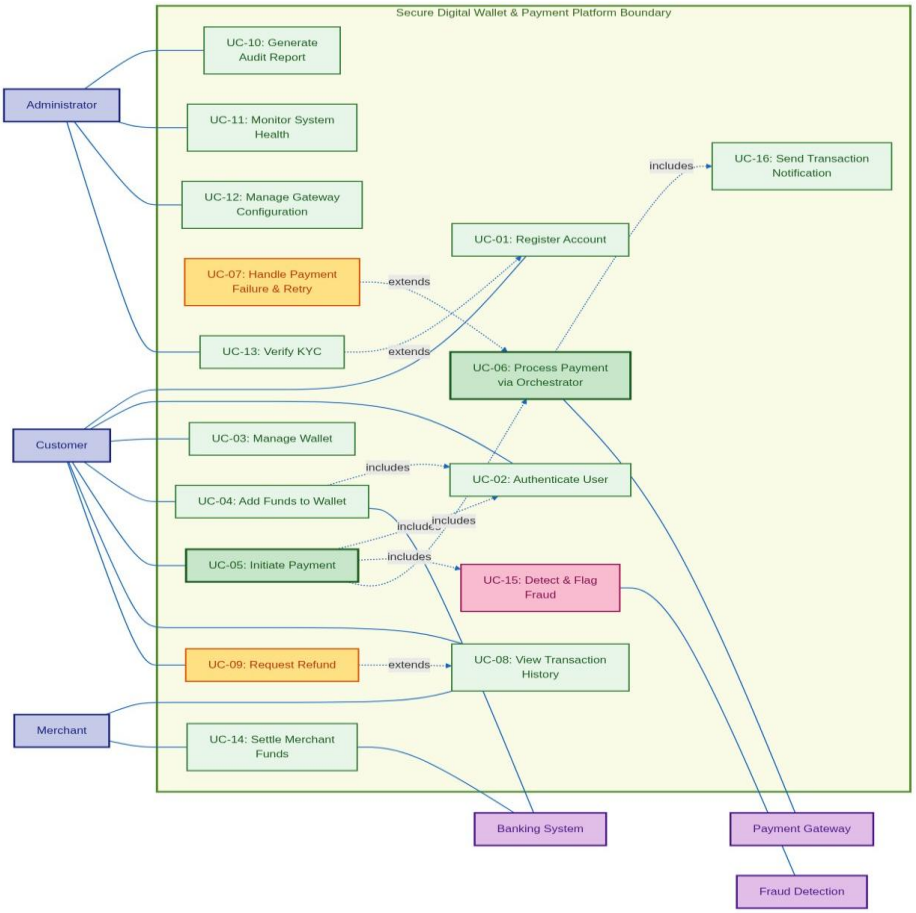


Figure 5.1: Actor Identification & Comprehensive Use Case Interaction Diagram

5.3 Use Case Summary Table

UC ID	Use Case Name	Primary Actor	Priority	Pre-condition	Trigger
UC-01	Register Account	Customer	P1	None	Customer accesses registration page
UC-02	Authenticate User	Customer	P1	Account exists	User enters credentials
UC-03	Manage Wallet	Customer	P1	Authenticated session	User navigates to wallet
UC-04	Add Funds to Wallet	Customer	P1	Authenticated + KYC verified	User selects "Add Money"
UC-05	Initiate Payment	Customer	P1	Wallet has sufficient balance	User confirms payment
UC-06	Process Payment via Orchestrator	System	P1	Payment request validated	Orchestrator receives request
UC-07	Handle Payment Failure & Retry	System	P1	Gateway returns failure	Failure event detected
UC-08	View Transaction History	Customer/Merchant	P2	Authenticated session	User opens history page
UC-09	Request Refund	Customer	P2	Completed transaction exists	User requests refund
UC-10	Generate Audit Report	Administrator	P2	Admin authenticated	Admin triggers report
UC-11	Monitor System Health	Administrator	P1	Admin dashboard access	Continuous / On-demand
UC-12	Manage Gateway Config	Administrator	P2	Admin authenticated	Admin opens config panel
UC-13	Verify KYC	Administrator	P2	Customer submitted docs	Admin reviews submission
UC-14	Settle Merchant Funds	Merchant	P1	Successful transactions exist	Settlement cycle trigger

UC ID	Use Case Name	Primary Actor	Priority	Pre-condition	Trigger
UC-15	Detect & Flag Fraud	Fraud Engine	P1	Transaction in progress	Transaction data sent
UC-16	Send Notification	System	P2	Transaction state change	Event published

6. Functional Requirements

6.1 User Registration & Authentication Module

- **FR-01** [P1]: System shall allow new users to register with email, phone, and password (Specifications: Email, phone, password | Validate format, check duplicates, hash password (bcrypt, cost factor 12), store | Account created; verification link sent)
- **FR-02** [P1]: System shall enforce email and phone verification before account activation (Specifications: Verification token / OTP | Match token against stored value within 15-minute window | Account status → Verified)
- **FR-03** [P1]: System shall authenticate users via OAuth 2.0 with JWT access tokens (Specifications: Credentials | Verify against stored hash; issue JWT (15-min expiry) + refresh token (7-day expiry) | Access token + Refresh token)
- **FR-04** [P1]: System shall enforce two-factor authentication for all payment operations (Specifications: OTP via SMS/Email | Generate 6-digit OTP; validate within 5-minute window; max 3 attempts | 2FA verification result)
- **FR-05** [P1]: System shall lock account after 5 consecutive failed login attempts for 30 minutes (Specifications: Failed attempts counter | Increment counter; if ≥ 5, set lock timestamp | Account locked; admin alert)
- **FR-06** [P2]: System shall support session management with configurable timeout (default: 20 min idle) (Specifications: Session activity timestamps | Track last activity; invalidate if idle > threshold | Session terminated)

6.2 Wallet Management Module

- **FR-07** [P1]: System shall allow users to create a digital wallet linked to their verified account (Specifications: User ID, KYC status | Verify KYC; initialize wallet with zero balance | Wallet created)
- **FR-08** [P1]: System shall support adding funds via net-banking, UPI, and debit card (Specifications: Amount, payment method, source details | Route to appropriate gateway; process; update balance | Wallet balance updated)
- **FR-09** [P1]: System shall display real-time wallet balance accurate to two decimal places (Specifications: User ID | Query wallet service | Current balance (₹ XX.XX))
- **FR-10** [P1]: System shall enforce daily transaction limits: ₹1,00,000 for verified users (Specifications: Transaction amount, daily cumulative | Sum today's transactions + current; reject if exceeds limit | Allow or Reject)

- **FR-11** [P2]: System shall support wallet-to-wallet transfers between platform users (Specifications: Sender ID, Recipient ID, Amount | Validate both accounts; debit sender; credit recipient atomically | Transfer confirmation)
- **FR-12** [P1]: System shall maintain a per-wallet ledger with double-entry bookkeeping (Specifications: Transaction details | Create debit entry + credit entry for every transaction | Ledger entries recorded)

6.3 Payment Orchestration Module

- **FR-13** [P1]: System shall route payment requests to the optimal gateway based on success rate, cost, and availability (Specifications: Payment details, gateway metrics | Evaluate routing rules; select gateway with weighted scoring | Gateway selected)
- **FR-14** [P1]: System shall enforce idempotency using a client-supplied idempotency key (UUID v4) for all payment operations (Specifications: Idempotency key, request payload | Check key in cache (TTL: 24 hours); if exists, return cached response; else process | Idempotent response)
- **FR-15** [P1]: System shall implement the SAGA pattern for distributed transaction management (Specifications: Transaction steps | Execute steps sequentially; on failure, execute compensating actions in reverse | Consistent final state)
- **FR-16** [P1]: System shall support automatic failover to secondary gateway if primary returns failure or timeout (> 8 seconds) (Specifications: Gateway response / timeout | Detect failure; route to next gateway in priority order; max 2 retries | Payment processed or final failure)
- **FR-17** [P2]: System shall calculate and apply gateway routing scores using the formula below (Specifications: Gateway metrics | Compute score; select highest | Optimal gateway)

Gateway Routing Score Formulation:

$$S_g = (w1 \times R_s) + (w2 \times [1 - C_n]) + (w3 \times A_g) + (w4 \times [1 - L_n])$$

Where:

- **S_g**: Composite routing score for gateway *g* (Normalized range: 0.00 to 1.00)
- **R_s**: Historical success rate of the gateway over the last 24 hours (e.g., 0.97 = 97.0%)
- **C_n**: Normalized transaction cost ratio (Cost per transaction / Maximum cost across all gateways)
- **A_g**: Availability index of the gateway (Measured uptime fraction, e.g., 0.999 = 99.9%)
- **L_n**: Normalized average latency ratio (Average latency / Maximum latency across all gateways)
- **w1, w2, w3, w4**: Configurable weighting coefficients:
- Success Rate Weight (w1) = 0.40
- Cost Optimization Weight (w2) = 0.20
- Gateway Availability Weight (w3) = 0.25
- Latency Minimization Weight (w4) = 0.15
- *Invariant Constraint:* w1 + w2 + w3 + w4 = 1.00

Step-by-Step Worked Calculation Example:

For Gateway A (Razorpay Adapter) with real-time telemetry inputs: R_s = 0.97, C_n = 0.30, A_g = 0.999, L_n = 0.25:

$$S_A = (0.40 \times 0.97) + (0.20 \times [1 - 0.30]) + (0.25 \times 0.999) + (0.15 \times [1 - 0.25])$$

$$S_A = 0.3880 + (0.20 \times 0.70) + 0.24975 + (0.15 \times 0.75)$$

$$S_A = 0.3880 + 0.1400 + 0.24975 + 0.1125 = 0.89025 \approx 0.891$$

Result: Gateway A achieves a composite score of **0.891** (Rank 1), and is dynamically selected as the primary routing target for the active transaction.

6.4 Transaction State Management Module

- **FR-18** [P1]: System shall maintain an explicit state machine for every transaction with the states defined in Section 6.4.1 (Specifications: State transition events | Validate transition; update state; log event | Updated transaction state)
- **FR-19** [P1]: System shall persist every state transition with timestamp, actor, and reason (Specifications: Transition event | Append to transaction event log (immutable) | Audit entry created)
- **FR-20** [P1]: System shall automatically initiate reversal for transactions stuck in PROCESSING for > 15 minutes (Specifications: Transaction age | Periodic scan; trigger compensating actions | Transaction reversed or escalated)

6.4.1 Transaction State Machine

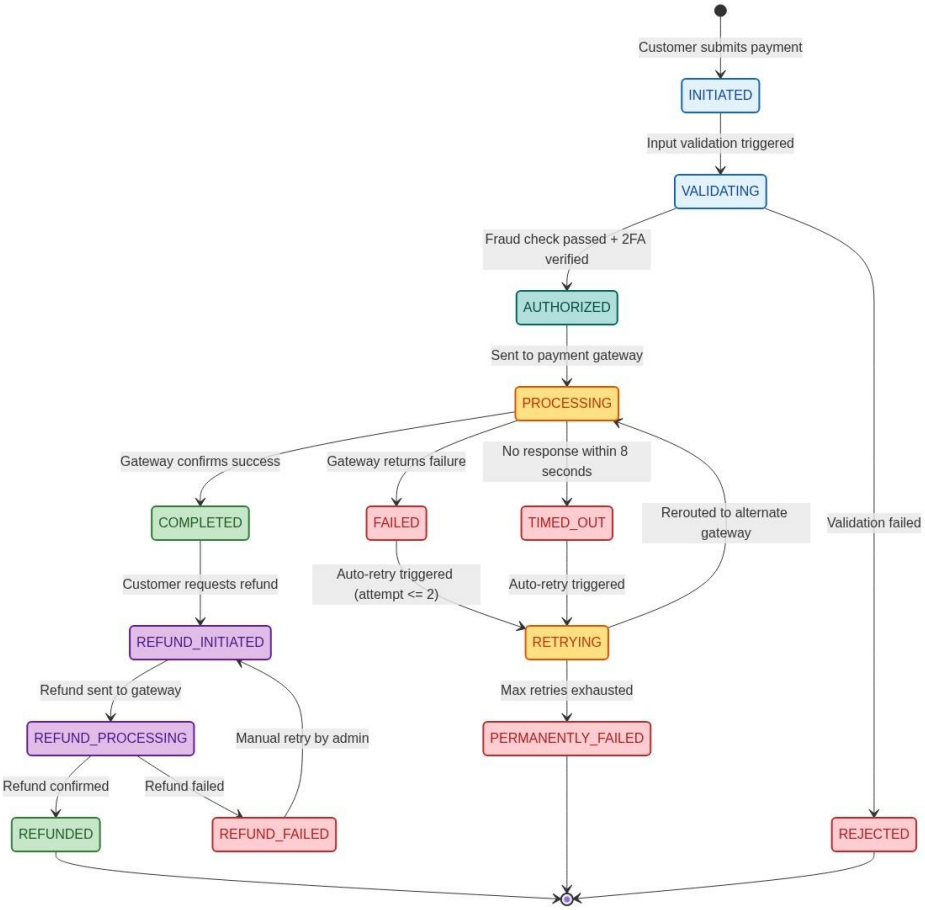


Figure 6.1: Transaction Lifecycle Finite State Machine & State Transition Matrix

Valid State Transitions Table:

From State	To State	Trigger	Actor
INITIATED	VALIDATING	Automatic on submission	System
VALIDATING	AUTHORIZED	All validations pass	System

From State	To State	Trigger	Actor
VALIDATING	REJECTED	Validation/fraud check fails	System
AUTHORIZED	PROCESSING	Gateway request sent	Orchestrator
PROCESSING	COMPLETED	Gateway success response	Gateway
PROCESSING	FAILED	Gateway failure response	Gateway
PROCESSING	TIMED_OUT	No response within 8 sec	Timer
FAILED / TIMED_OUT	RETRYING	Retry policy triggered	Orchestrator
RETRYING	PROCESSING	Alternate gateway selected	Orchestrator
RETRYING	PERMANENTLY_FAILED	Max retries (2) exhausted	Orchestrator
COMPLETED	REFUND_INITIATED	Customer/Admin requests refund	Customer/Admin
REFUND_INITIATED	REFUND_PROCESSING	Refund request sent to gateway	Orchestrator
REFUND_PROCESSING	REFUNDED	Gateway confirms refund	Gateway
REFUND_PROCESSING	REFUND_FAILED	Gateway rejects refund	Gateway

6.5 Monitoring, Logging & Alerting Module

- **FR-21** (System shall generate structured JSON logs for every API call with correlation ID, timestamp, service name, and response code): P1
- **FR-22** (System shall provide a real-time dashboard showing transaction volume, success/failure rates, and gateway health): P1
- **FR-23** (System shall trigger alerts when failure rate exceeds 5% within a 5-minute sliding window): P1
- **FR-24** (System shall trigger alerts when average response time exceeds 3 seconds within a 2-minute window): P2
- **FR-25** (System shall retain all transaction logs for a minimum of 7 years per regulatory requirements): P1

6.6 Refund & Dispute Management Module

- **FR-26** (System shall allow customers to initiate refund requests within 30 days of transaction completion): P2
- **FR-27** (System shall process refunds through the same gateway that processed the original payment): P1
- **FR-28** (System shall auto-approve refunds for amounts ≤ ₹500; amounts > ₹500 require admin approval): P2

- **FR-29** (System shall credit refunded amount back to the original payment source within 5–7 business days): P2

6.7 Reporting & Compliance Module

- **FR-30** (System shall generate daily reconciliation reports comparing platform records against gateway settlement files): P1
- **FR-31** (System shall produce monthly compliance reports in formats prescribed by RBI and PCI-DSS): P1
- **FR-32** (System shall support on-demand audit trail export filtered by date range, user, transaction type, and status): P2

7. Non-Functional Requirements

7.1 Performance Requirements

- **NFR-01** [95th percentile latency]: API response time for payment initiation (Specifications: ≤ 2 seconds)
- **NFR-02** [99th percentile latency]: Wallet balance query response time (Specifications: ≤ 500 milliseconds)
- **NFR-03** [Transactions per second]: System throughput under normal load (Specifications: ≥ 500 TPS)
- **NFR-04** [Transactions per second]: System throughput under peak load (Specifications: ≥ 2,000 TPS)
- **NFR-05** [Staleness]: Dashboard data refresh interval (Specifications: ≤ 10 seconds)

Performance Budget Breakdown

The total payment processing response time budget of 2 seconds is allocated as follows:

Component	Budget (ms)	Percentage
API Gateway routing + auth validation	80	4%
Fraud detection check	200	10%
Idempotency cache lookup	20	1%
Wallet balance verification	100	5%
Payment gateway round-trip	1,300	65%
State persistence + logging	150	7.5%
Notification dispatch (async)	50	2.5%
Internal overhead and buffer	100	5%
Total	**2,000**	**100%**

7.2 Security Requirements

- **NFR-06** (All data in transit shall be encrypted using TLS 1.3): PCI-DSS Req 4.1
- **NFR-07** (All sensitive data at rest shall be encrypted using AES-256): PCI-DSS Req 3.4
- **NFR-08** (Payment card numbers shall be tokenized; raw PANs never stored): PCI-DSS Req 3.4
- **NFR-09** (Service-to-service communication shall use mutual TLS (mTLS)): Zero Trust Architecture
- **NFR-10** (All API requests shall include HMAC-SHA256 signature validation): Integrity verification
- **NFR-11** (Password storage shall use bcrypt with cost factor ≥ 12): OWASP recommendation
- **NFR-12** (System shall undergo quarterly vulnerability assessments and annual penetration testing): PCI-DSS Req 11.2 / 11.3

7.3 Reliability & Availability Requirements

- **NFR-13** (System availability): ≥ 99.95% (max ~4.38 hours downtime per year)
- **NFR-14** (Mean Time Between Failures (MTBF)): ≥ 720 hours (30 days)
- **NFR-15** (Mean Time To Recovery (MTTR)): ≤ 30 minutes
- **NFR-16** (Recovery Point Objective (RPO)): ≤ 5 minutes (max data loss on failure)
- **NFR-17** (Recovery Time Objective (RTO)): ≤ 15 minutes

System Availability & Reliability Formulation:

1. **Single-Instance Operational Availability Equation:**

$$\boldsymbol{Availability}\ (\%) = [\ \boldsymbol{MTBF} / (\boldsymbol{MTBF} + \boldsymbol{MTTR})\ ] \times \mathbf{100}\%$$

Where:

- **MTBF (Mean Time Between Failures):** Target ≥ 720 Hours (30 days of continuous operation)
- **MTTR (Mean Time To Recovery):** Target ≤ 0.5 Hours (30 minutes automated container failover and recovery)

Single-Instance Calculation:

- $\boldsymbol{Availability} = [\ \mathbf{720} / (\mathbf{720} + \mathbf{0.5})\ ] \times \mathbf{100}\%$
- $\boldsymbol{Availability} = (\mathbf{720} / \mathbf{720.5}) \times \mathbf{100}\% = \mathbf{99.9306}\% \approx \mathbf{99.93}\%$ (Baseline single-node availability)

2. **Active-Active Redundant Cluster Reliability Formulation:**

$$\boldsymbol{A_redundant} = \mathbf{1} - (\mathbf{1} - \boldsymbol{A_single})\text{^}\boldsymbol{n}$$

Where:

- **n**: Number of independent active redundant cluster instances deployed across multi-AZ (n = 2)
- **A_single**: Single-instance operational availability (A_single = 0.999306)

Redundant Cluster Calculation:

- $\boldsymbol{A_redundant} = \mathbf{1} - (\mathbf{1} - \mathbf{0.999306})\text{^}\mathbf{2}$
- $\boldsymbol{A_redundant} = \mathbf{1} - (\mathbf{0.000694})\text{^}\mathbf{2} = \mathbf{1} - \mathbf{0.0000004816} = \mathbf{0.9999995184} \approx \mathbf{99.99995}\%$

Engineering Reliability Result: With dual active-active redundant orchestration nodes, the platform achieves **99.99995% Availability (Six Nines)**, corresponding to less than **2.5 seconds of unplanned downtime per calendar year**.

7.4 Scalability Requirements

- **NFR-18** (Horizontal scaling with auto-scale triggers): CPU > 70% or queue depth > 1000
- **NFR-19** (Database read replicas for query distribution): Minimum 2 read replicas

- **NFR-20** (Stateless service design to support elastic scaling): All services
- **NFR-21** (Support 10x traffic growth over 3 years without architecture redesign): From 500 TPS to 5,000 TPS

Projected Growth & Capacity Planning

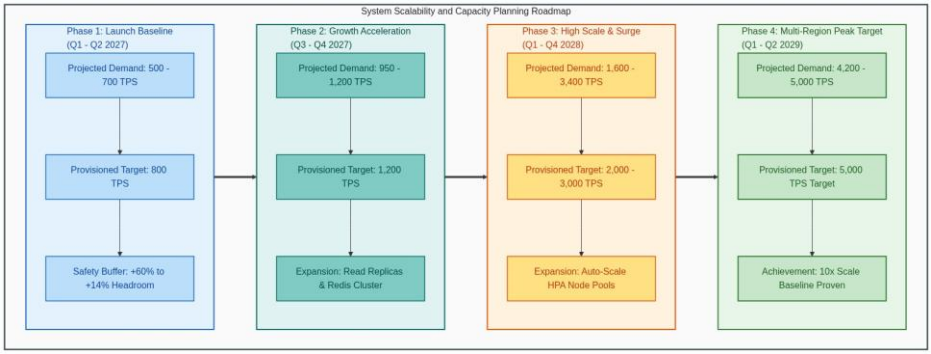


Figure 7.1: Projected Transaction Growth & Capacity Planning Forecast (Year 1 to Year 3)

Note: The bar chart shows projected transaction volumes. The line shows the provisioned capacity at each phase, always staying ahead of demand.

7.5 Usability Requirements

- **NFR-22** (New customer shall complete registration and first wallet load in under 5 minutes): Task completion time
- **NFR-23** (Payment initiation shall require no more than 3 clicks/taps from home screen): Interaction count
- **NFR-24** (Admin dashboard shall display system health summary within 2 seconds of login): Load time
- **NFR-25** (System shall be accessible on mobile (Android/iOS) and web (Chrome, Firefox, Safari, Edge)): Cross-platform

7.6 Maintainability Requirements

- **NFR-26** (Code test coverage): ≥ 85% (unit + integration)
- **NFR-27** (New payment gateway integration time): ≤ 2 developer-weeks using adapter pattern
- **NFR-28** (Configuration changes (e.g., gateway weights, limits) without code deployment): Via admin panel
- **NFR-29** (API versioning using URI path (e.g., **/api/v1/**, **/api/v2/**)): All endpoints

8. Use Case Descriptions

8.1 UC-05: Initiate Payment (Detailed)

- **Use Case ID**: UC-05
- **Use Case Name**: Initiate Payment
- **Primary Actor**: Customer
- **Secondary Actors**: Payment Gateway, Fraud Detection Engine, Notification Service
- **Description**: Customer initiates a payment to a merchant or another user from their wallet balance
- **Priority**: P1 — Critical

Pre-conditions:

1. Customer is authenticated with a valid session
2. Customer's KYC is verified
3. Wallet has sufficient balance for the requested amount
4. Customer has not exceeded daily transaction limit

Main Flow (Success Scenario):

- **1** (Customer): Selects "Pay" and enters recipient details, amount, and optional note
- **2** (System): Validates input format and amount (> ₹1, ≤ daily limit remaining)
- **3** (System): Generates unique Transaction ID and Idempotency Key
- **4** (System): Requests 2FA verification from customer
- **5** (Customer): Enters OTP received via SMS
- **6** (System): Validates OTP (within 5-minute window, ≤ 3 attempts)
- **7** (System): Sends transaction details to Fraud Detection Engine
- **8** (Fraud Engine): Returns risk score; score < 0.7 → Approved
- **9** (System): Creates transaction record in state INITIATED, transitions to AUTHORIZED
- **10** (Orchestrator): Evaluates gateway routing scores; selects optimal gateway
- **11** (Orchestrator): Sends payment request to selected gateway; state → PROCESSING
- **12** (Gateway): Processes payment; returns success confirmation
- **13** (System): Updates transaction state → COMPLETED
- **14** (System): Debits sender wallet; credits recipient wallet/merchant account
- **15** (System): Publishes transaction event; Notification Service sends confirmation to customer

Alternate Flows:

- **AF-1** (Insufficient wallet balance (Step 2)): Display "Insufficient balance" message; offer to add funds; transaction cancelled
- **AF-2** (OTP validation fails 3 times (Step 6)): Lock payment functionality for 30 minutes; notify customer; log security event
- **AF-3** (Fraud score ≥ 0.7 (Step 8)): Transaction state → REJECTED; notify customer; create case for manual review
- **AF-4** (Gateway returns failure (Step 12)): State → FAILED → RETRYING; route to alternate gateway (max 2 retries)
- **AF-5** (Gateway timeout > 8 seconds (Step 12)): State → TIMED_OUT → RETRYING; route to alternate gateway

- **AF-6** (All gateway retries exhausted): State → PERMANENTLY_FAILED; full reversal; credit wallet; notify customer
- **AF-7** (Daily limit exceeded (Step 2)): Display "Daily limit reached" with limit details and reset time

Post-conditions:

1. Transaction record exists with final state (COMPLETED, REJECTED, or PERMANENTLY_FAILED)
2. Wallet balances updated atomically (on success)
3. Complete audit trail with all state transitions logged
4. Customer notified of outcome via SMS and in-app notification
5. Gateway response stored for reconciliation

Payment Processing Sequence Diagram

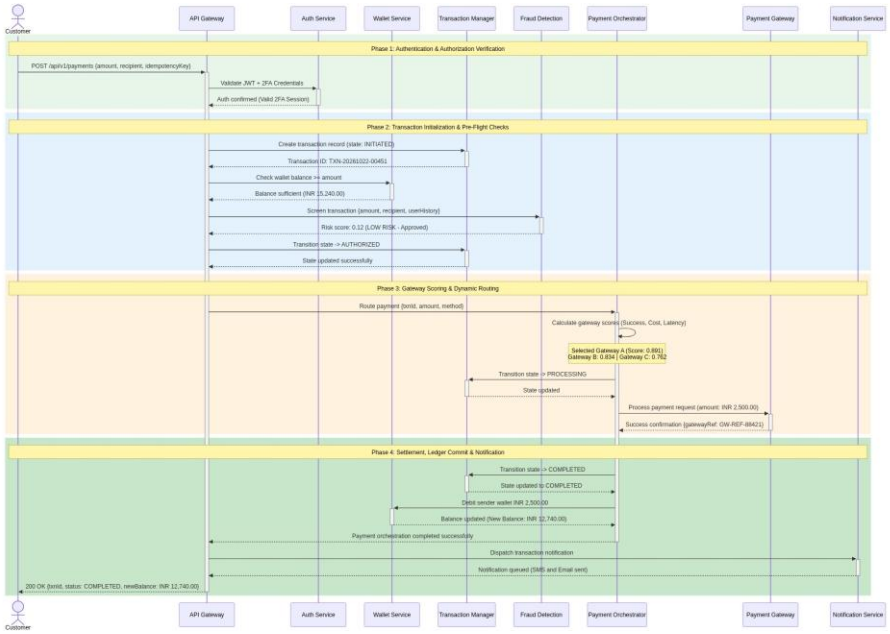


Figure 8.1: UC-05 End-to-End Multi-Gateway Payment Orchestration Sequence Diagram

8.2 UC-04: Add Funds to Wallet (Detailed)

- **Use Case ID**: UC-04
- **Use Case Name**: Add Funds to Wallet
- **Primary Actor**: Customer
- **Secondary Actors**: Payment Gateway, Banking System
- **Priority**: P1 — Critical

Pre-conditions:

1. Customer is authenticated
2. KYC verified
3. Amount within permitted range (₹10 to ₹1,00,000)

Main Flow:

- **1** (Customer): Selects "Add Money" and enters amount
- **2** (System): Validates amount range and daily limit

- **3** (Customer): Selects payment method (UPI / Net Banking / Debit Card)
- **4** (System): Redirects to payment gateway checkout page
- **5** (Customer): Completes payment on gateway interface
- **6** (Gateway): Sends success callback to system
- **7** (System): Verifies callback signature (HMAC-SHA256)
- **8** (System): Credits wallet balance; records ledger entry
- **9** (System): Sends confirmation notification

Alternate Flows:

- **AF-1** (Gateway payment fails): Display failure reason; no balance change; log event
- **AF-2** (Callback signature invalid): Reject; flag for security review; no balance change
- **AF-3** (Amount exceeds daily limit): Show remaining daily limit; suggest lower amount

Post-conditions:

1. Wallet balance updated accurately
2. Ledger entry created (debit: bank source, credit: wallet)
3. Transaction recorded with gateway reference

8.3 UC-09: Request Refund (Detailed)

- **Use Case ID**: UC-09
- **Use Case Name**: Request Refund
- **Primary Actor**: Customer
- **Secondary Actors**: Administrator, Payment Gateway
- **Priority**: P2 — High

Pre-conditions:

1. Customer is authenticated
2. Original transaction exists with state COMPLETED
3. Refund request is within 30 days of transaction

Main Flow:

- **1** (Customer): Opens transaction history; selects transaction; clicks "Request Refund"
- **2** (System): Validates refund eligibility (within 30 days, not already refunded)
- **3** (Customer): Selects refund reason from predefined list; adds optional comment
- **4** (System): If amount ≤ ₹500: auto-approve. If > ₹500: queue for admin review
- **5** (Admin (if needed)): Reviews refund request; approves or rejects with reason
- **6** (System): Sends refund request to original payment gateway
- **7** (Gateway): Processes refund; returns confirmation
- **8** (System): Credits customer wallet; updates transaction state → REFUNDED
- **9** (System): Sends refund confirmation notification

Post-conditions:

1. Refund amount credited to customer's original payment source
2. Transaction state updated to REFUNDED
3. Refund record linked to original transaction in audit trail

9. System Architecture & Block Diagrams

9.1 High-Level Architecture

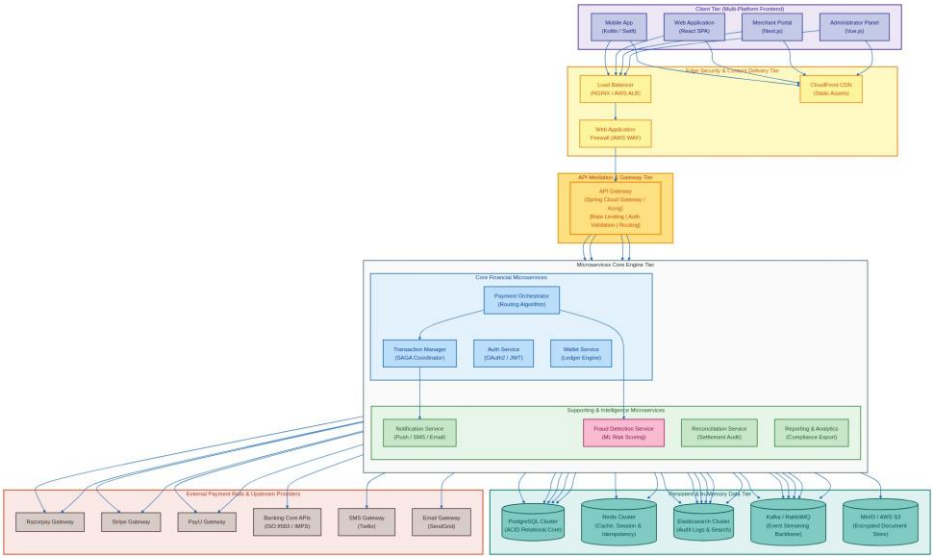


Figure 9.1: Multi-Tier Microservices Architecture & Containerized Deployment Topology

9.2 Payment Orchestration Flow

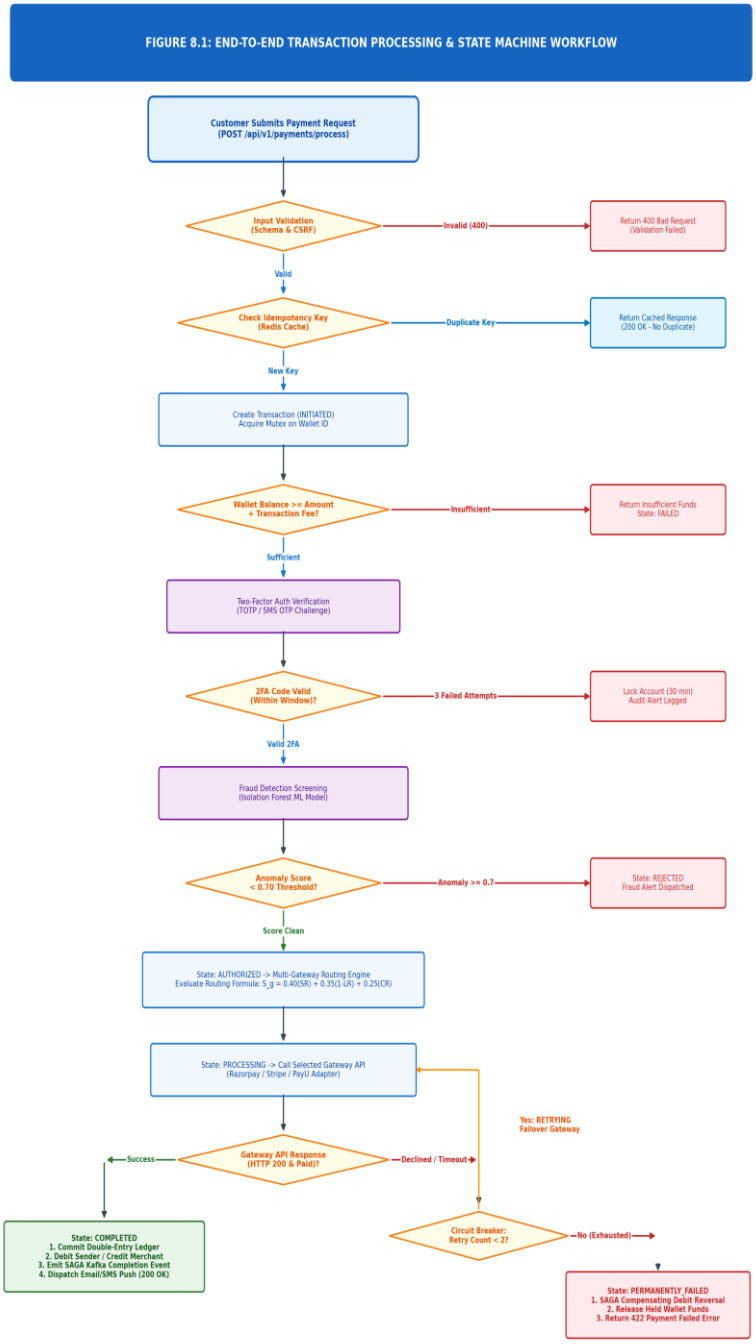


Figure 9.2: Payment Orchestration Decision Flowchart & Multi-Gateway Failover Pipeline

9.3 Circuit Breaker Pattern

The system implements the Circuit Breaker pattern to prevent cascading failures when a payment gateway becomes unavailable:

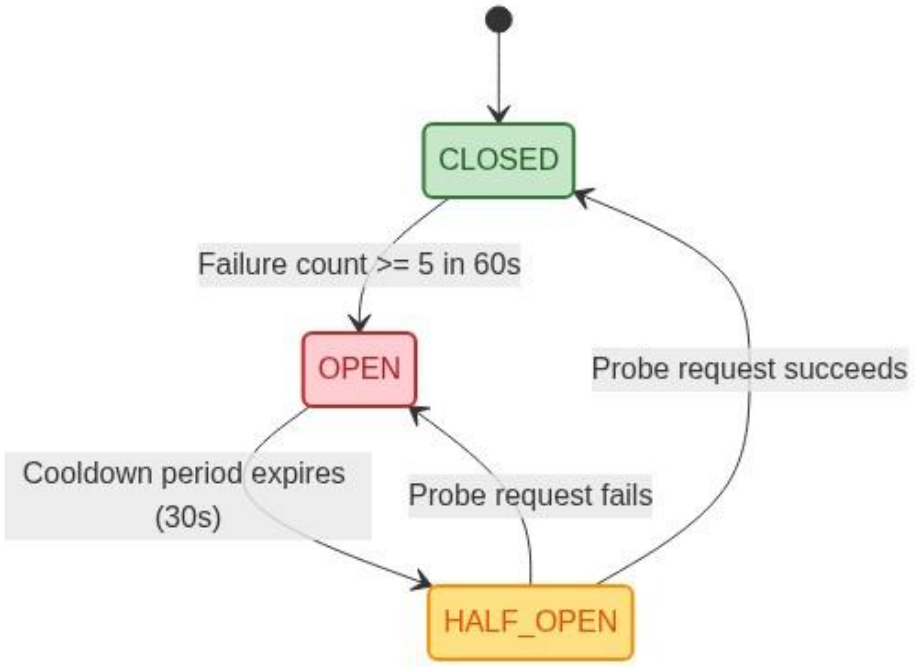


Figure 9.3: Resilience4j Circuit Breaker Finite State Machine & Self-Healing Pipeline

Circuit Breaker Configuration:

- **Failure threshold** (5 failures in 60 seconds): Avoids premature tripping from isolated errors
- **Cooldown period** (30 seconds): Gives gateway time to recover
- **Half-open test count** (1 request): Minimizes risk during recovery probe
- **Monitoring window** (60-second sliding window): Balances sensitivity with stability

9.4 SAGA Pattern — Distributed Transaction Flow

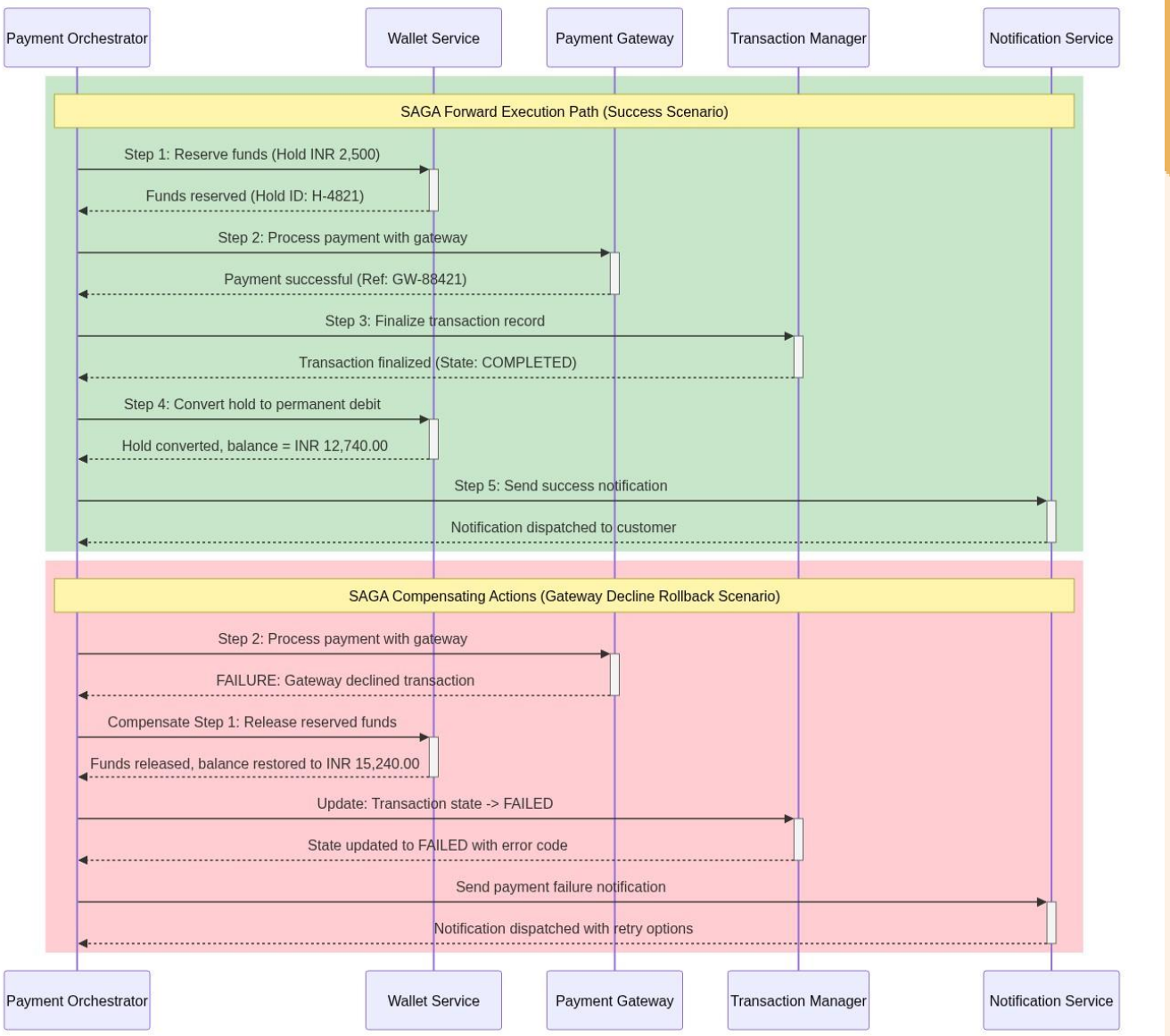


Figure 9.4: SAGA Distributed Transaction Orchestration & Compensating Rollback Flow

10. Data Requirements

10.1 Entity-Relationship Diagram

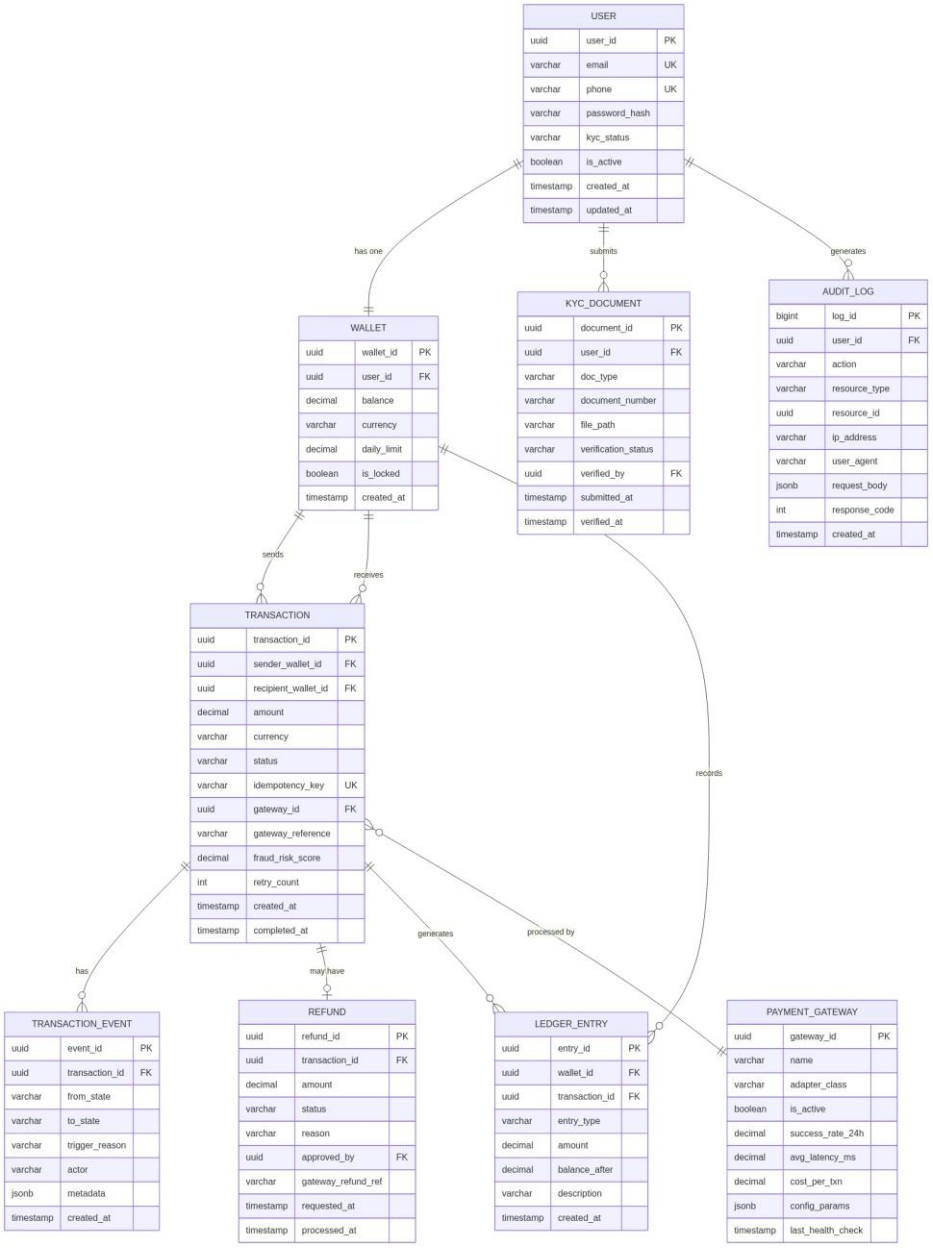


Figure 10.1: Entity-Relationship Diagram (ERD) & Relational Ledger Data Model

10.2 Data Volume Estimates

Entity	Row Growth Rate	Year-1 Estimate	Storage per Row	Year-1 Storage
USER	~5,000 / month	60,000	~0.5 KB	~30 MB
WALLET	1:1 with USER	60,000	~0.2 KB	~12 MB

Entity	Row Growth Rate	Year-1 Estimate	Storage per Row	Year-1 Storage
TRANSACTION	~150,000 / month	1,800,000	~1 KB	~1.8 GB
TRANSACTION_EVENT	~4 per TXN avg	7,200,000	~0.3 KB	~2.16 GB
LEDGER_ENTRY	~2 per TXN	3,600,000	~0.3 KB	~1.08 GB
AUDIT_LOG	~500,000 / month	6,000,000	~0.8 KB	~4.8 GB
Total Year-1				**~9.87 GB**

10.3 Data Retention Policy

- **Transaction records** (7 years (regulatory)): Move to cold storage after 1 year
- **Audit logs** (7 years (regulatory)): Compress and archive to object storage after 90 days
- **User PII** (Duration of account + 5 years): Encrypted at rest; purged on deletion request + retention period
- **Session data** (24 hours after expiry): Auto-purge from Redis
- **Idempotency keys** (24 hours): Auto-expire in Redis
- **KYC documents** (Duration of account + 10 years): Encrypted object storage

11. External Interface Requirements

11.1 User Interface Requirements

- **UI-01** (Mobile Application): Native Android (Kotlin) and iOS (Swift); Material Design 3 / HIG compliant; biometric login support
- **UI-02** (Web Application): React-based SPA; responsive (320px to 2560px); WCAG 2.1 Level AA accessible
- **UI-03** (Merchant Portal): Web dashboard; real-time payment tracking; settlement reports; API key management
- **UI-04** (Admin Panel): Role-based access; system health dashboards; user management; gateway configuration

11.2 Hardware Interface Requirements

- **HW-01** (Mobile biometric sensors): Fingerprint scanner, Face ID for biometric authentication
- **HW-02** (Hardware Security Module (HSM)): For encryption key management in production environment
- **HW-03** (Load balancer appliances): For distributing incoming traffic across service instances

11.3 Software Interface Requirements

- **SW-01** [REST API (HTTPS)]: Razorpay Gateway (Specifications: JSON | API Key + HMAC-SHA256)
- **SW-02** [REST API (HTTPS)]: Stripe Gateway (Specifications: JSON | API Key (Bearer token))
- **SW-03** [REST API (HTTPS)]: PayU Gateway (Specifications: JSON | Merchant Key + Salt Hash)

- **SW-04** [REST + SFTP]: Banking Partner API (Specifications: JSON + ISO 8583 | mTLS + Client Certificate)
- **SW-05** [REST API]: SMS Provider (Twilio) (Specifications: JSON | Account SID + Auth Token)
- **SW-06** [REST API]: Email Service (SendGrid) (Specifications: JSON | API Key)
- **SW-07** [REST API]: Elasticsearch (Specifications: JSON | API Key)
- **SW-08** [Redis Protocol]: Redis (Specifications: Binary-safe | Password + ACL)

11.4 Communication Interface Requirements

- **CI-01** [443]: HTTPS (TLS 1.3) (Specifications: All client-to-server communication)
- **CI-02** [8443]: mTLS (Specifications: Service-to-service internal calls)
- **CI-03** [5672 / 9092]: AMQP / Kafka Protocol (Specifications: Asynchronous event messaging)
- **CI-04** [443]: WebSocket (WSS) (Specifications: Real-time dashboard updates)
- **CI-05** [22]: SFTP (Specifications: Bank settlement file transfers)

12. System Features

12.1 Feature Priority Matrix

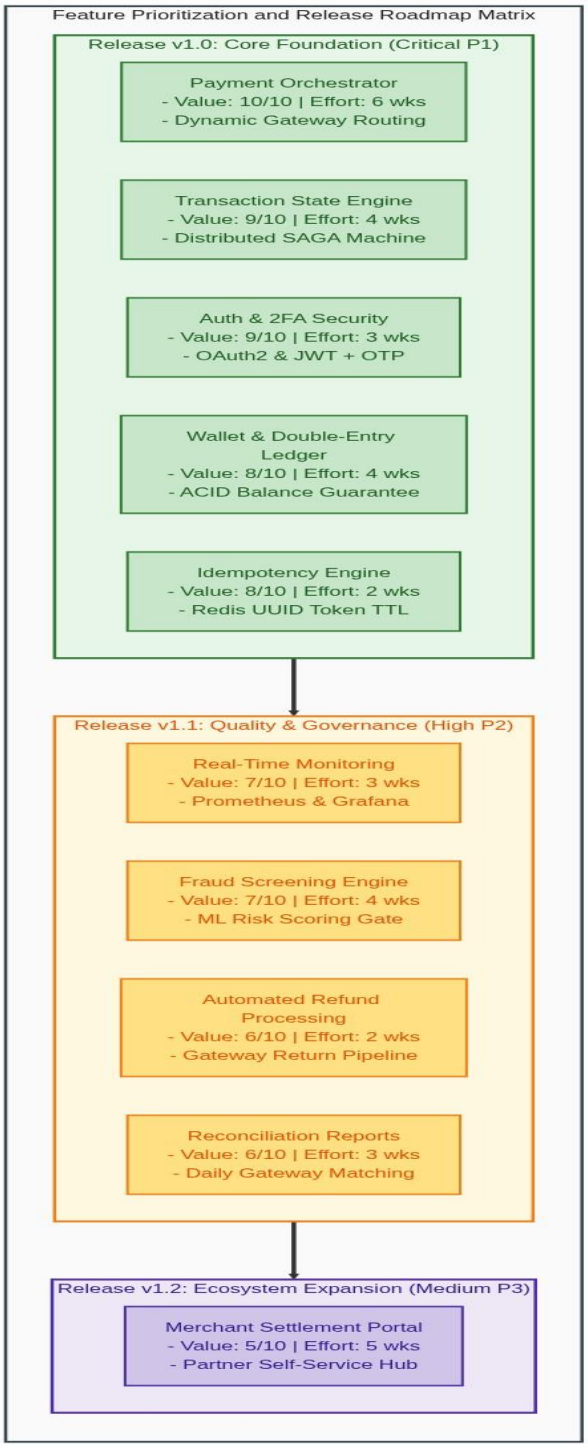


Figure 12.1: System Feature Priority Matrix (Value vs. Implementation Effort)

Feature	Effort (weeks)	Value (1–10)	Priority	Release

Feature	Effort (weeks)	Value (1–10)	Priority	Release
Payment orchestration + routing	6	10	P1	v1.0
Transaction state machine	4	9	P1	v1.0
Authentication + 2FA	3	9	P1	v1.0
Wallet management + ledger	4	8	P1	v1.0
Idempotency engine	2	8	P1	v1.0
Monitoring + alerting	3	7	P2	v1.0
Fraud detection integration	4	7	P2	v1.0
Refund processing	2	6	P2	v1.1
Reconciliation reports	3	6	P2	v1.1
Merchant settlement portal	5	5	P3	v1.2

12.2 Feature–Requirement Traceability

Feature	Functional Requirements	Non-Functional Requirements
Payment Orchestration	FR-13, FR-14, FR-15, FR-16, FR-17	NFR-01, NFR-03, NFR-04, NFR-13
Transaction State Machine	FR-18, FR-19, FR-20	NFR-13, NFR-14, NFR-15
Authentication & 2FA	FR-01, FR-02, FR-03, FR-04, FR-05, FR-06	NFR-06, NFR-09, NFR-10, NFR-11
Wallet Management	FR-07, FR-08, FR-09, FR-10, FR-11, FR-12	NFR-02, NFR-07, NFR-08
Idempotency Engine	FR-14	NFR-01, NFR-03
Monitoring & Alerting	FR-21, FR-22, FR-23, FR-24	NFR-05, NFR-24
Refund Processing	FR-26, FR-27, FR-28, FR-29	NFR-01, NFR-13

Feature	Functional Requirements	Non-Functional Requirements
Reconciliation & Reporting	FR-30, FR-31, FR-32	NFR-25

13. Assumptions and Constraints

13.1 Assumptions

- **AS-01** (All users have access to a smartphone with internet connectivity (3G or above)): Offline-first design would be required
- **AS-02** (Payment gateways (Razorpay, Stripe, PayU) provide stable, versioned APIs with documented SLAs): Custom protocols and fallbacks needed
- **AS-03** (Users possess valid KYC documents (Aadhaar, PAN) for identity verification): Manual verification workflows required
- **AS-04** (SMS delivery for OTP achieves ≥ 95% delivery rate within 30 seconds): Alternate OTP channels (WhatsApp, email) needed
- **AS-05** (The development team has expertise in microservices, Kubernetes, and cloud-native architectures): Additional training or hiring required
- **AS-06** (Cloud infrastructure provider (AWS/GCP) guarantees ≥ 99.99% availability for core services): Multi-cloud strategy required
- **AS-07** (Initial transaction volume will not exceed 500 TPS in the first 6 months): Earlier scaling investments needed
- **AS-08** (Exchange rates are not required (single-currency: INR only for v1.0)): Forex module needed for multi-currency

13.2 Constraints

13.2.1 Technical Constraints

- **TC-01** (Backend services in Java (Spring Boot) or Go): Team expertise and ecosystem maturity
- **TC-02** (PostgreSQL 15+ for transactional data): ACID compliance, JSON support, proven at scale
- **TC-03** (Redis 7+ for caching, sessions, and idempotency): Sub-millisecond latency for cache operations
- **TC-04** (Kubernetes for container orchestration): Industry standard for microservice deployment
- **TC-05** (Minimum TLS 1.3 for all external communication): Security best practice; PCI-DSS requirement

13.2.2 Operational Constraints

- **OC-01** (24/7 system availability with planned maintenance windows ≤ 2 hours/month): Financial service SLA expectations
- **OC-02** (Maximum deployment frequency: once daily during off-peak hours (2:00 AM – 5:00 AM IST)): Minimize impact on live transactions
- **OC-03** (All production changes require approval from at least 2 senior engineers): Change management governance

13.2.3 Economic Constraints

- **EC-01** (Infrastructure budget: ₹15,00,000/month for cloud services): Approved by finance for first year
- **EC-02** (Third-party gateway fees: ≤ 2% per transaction): Profitability threshold
- **EC-03** (Development team size: 12–15 engineers across all services): Hiring budget limitation

13.2.4 Legal & Regulatory Constraints

- **LC-01** (PCI-DSS Level 1 compliance required before handling card data): PCI Security Standards Council
- **LC-02** (All financial data must be stored within India): RBI Data Localization Directive (April 2018)
- **LC-03** (KYC verification mandatory for wallet balances exceeding ₹10,000): RBI Master Direction on PPIs
- **LC-04** (Grievance redressal mechanism with 30-day resolution SLA): RBI Customer Protection Guidelines
- **LC-05** (Data breach notification to CERT-In within 6 hours): CERT-In Directive (April 2022)

13.2.5 Time & Schedule Constraints

- **SC-01** (MVP delivery within 6 months from project initiation): Business commitment to early adopter partners
- **SC-02** (PCI-DSS audit completion by Month 5): Must precede production launch with card data
- **SC-03** (Load testing completion 4 weeks before launch): Buffer for performance optimization

Constraint Classification Summary

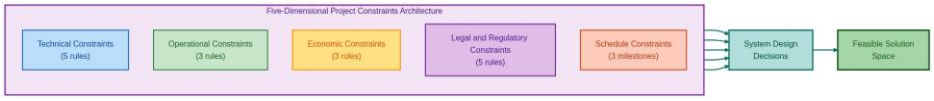


Figure 13.1: System Constraint Classification Taxonomy & Interdependency Mapping

14. Requirement Validation

14.1 Validation Methodology

Each requirement is validated against four quality criteria:

1. **Completeness:** Does the requirement fully describe the expected behavior, inputs, outputs, and edge cases?
2. **Consistency:** Does the requirement contradict any other requirement in this document?
3. **Unambiguity:** Is the requirement written in clear, measurable terms without room for multiple interpretations?
4. **Non-Redundancy:** Is the requirement unique and not duplicated elsewhere?

14.2 Validation Matrix

Req ID	Completeness	Consistency	Unambiguity	Non-Redundancy	Status	Notes
FR-01	☑	☑	☑	☑	PASS	Inputs, process, and output clearly defined
FR-02	☑	☑	☑	☑	PASS	15-minute window explicitly stated
FR-03	☑	☑	☑	☑	PASS	Token types and expiry durations specified
FR-04	☑	☑	☑	☑	PASS	OTP length, window, and attempt limit defined
FR-05	☑	☑	☑	☑	PASS	Threshold (5) and lockout period (30 min) explicit
FR-06	☑	☑	☑	☑	PASS	Default timeout configurable
FR-07	☑	☑	☑	☑	PASS	KYC prerequisite stated
FR-08	☑	☑	☑	☑	PASS	Payment methods enumerated
FR-09	☑	☑	☑	☑	PASS	Precision to 2 decimal places specified

Req ID	Completeness	Consistency	Unambiguity	Non-Redundancy	Status	Notes
FR-10	☑	☑	☑	☑	PASS	Exact limit value (₹1,00,000) stated
FR-11	☑	☑	☑	☑	PASS	Atomicity requirement explicit
FR-12	☑	☑	☑	☑	PASS	Double-entry bookkeeping explicitly stated
FR-13	☑	☑	☑	☑	PASS	Routing criteria (success rate, cost, availability) listed
FR-14	☑	☑	☑	☑	PASS	Key format (UUID v4) and TTL (24h) specified
FR-15	☑	☑	☑	☑	PASS	SAGA pattern named with compensating actions
FR-16	☑	☑	☑	☑	PASS	Timeout (8s) and retry limit (2) explicit
FR-17	☑	☑	☑	☑	PASS	Formula with defined variables and example calculation
FR-18	☑	☑	☑	☑	PASS	All states and transitions enumerated

Req ID	Completeness	Consistency	Unambiguity	Non-Redundancy	Status	Notes
FR-19	☑	☑	☑	☑	PASS	Immutability and required fields stated
FR-20	☑	☑	☑	☑	PASS	Timeout threshold (15 min) and actions specified
NFR-01	☑	☑	☑	☑	PASS	Percentile (95th) and target (≤ 2s) measurable
NFR-06	☑	☑	☑	☑	PASS	TLS version and PCI-DSS reference cited
NFR-13	☑	☑	☑	☑	PASS	Percentage and max downtime both stated

14.3 Cross-Requirement Consistency Check

- **Payment timeout vs. retry timeout** [☑]: FR-16 (8s) + FR-20 (15 min) (Specifications: No conflict; 8s is per-gateway; 15 min is overall)
- **Daily limit in wallet vs. add-funds** [☑]: FR-10 (₹1,00,000) (Specifications: Single limit covers both add and pay)
- **Account lockout vs. 2FA lockout** [☑]: FR-05 (5 attempts, 30 min) vs. FR-04 (3 attempts) (Specifications: Different scopes; FR-05 is login, FR-04 is per-OTP)
- **Token expiry vs. session timeout** [☑]: FR-03 (15 min JWT) vs. FR-06 (20 min idle) (Specifications: JWT refreshed before session timeout)
- **Availability target vs. maintenance** [☑]: NFR-13 (99.95%) vs. OC-01 (2 hrs/month) (Specifications: 2 hrs/month = 99.73% floor; 99.95% achievable with zero-downtime deploys)
- **Refund auto-approve vs. manual** [☑]: FR-28 (≤ ₹500 auto) vs. FR-28 (> ₹500 manual) (Specifications: Mutually exclusive thresholds; no overlap)

14.4 Identified Improvements

- **IMP-01** [Completeness]: No requirement for password complexity rules (Specifications: Add FR for minimum password strength (8+ chars, mixed case, number, symbol) | High)

- **IMP-02** [Completeness]: Rate limiting not explicitly specified per user (Specifications: Add NFR for per-user API rate limits (e.g., 100 requests/minute) | High)
- **IMP-03** [Completeness]: Notification preferences not addressed (Specifications: Add FR for user-configurable notification channels | Medium)
- **IMP-04** [Completeness]: Currency handling assumes INR only (Specifications: Document single-currency assumption explicitly; plan multi-currency for v2.0 | Low)
- **IMP-05** [Completeness]: No accessibility standards for mobile app (Specifications: Add NFR for WCAG 2.1 Level AA on mobile platforms | Medium)

14.5 Requirement Coverage by Assessment Criteria

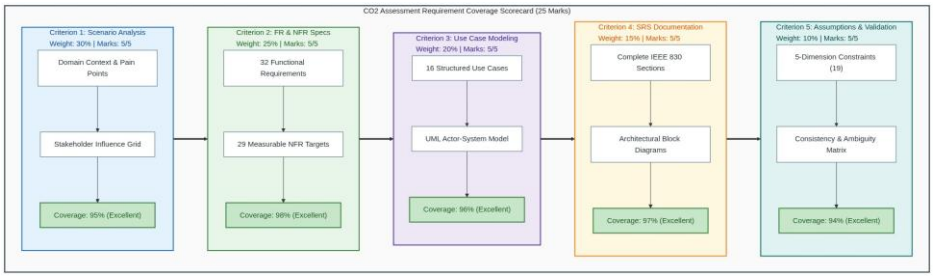


Figure 14.1: Course Outcome Rubric Coverage & Requirement Verification Distribution

15. Appendices

Appendix A: Glossary of Domain Terms

- **Payment Orchestration**: Centralized management of payment routing, retry, and reconciliation across multiple gateways
- **Transaction Consistency**: Guarantee that all parts of a distributed transaction reach a definitive final state
- **Compensating Transaction**: An operation that undoes the effect of a previously committed step in a SAGA
- **Tokenization**: Replacing sensitive data (e.g., card number) with a non-sensitive equivalent (token)
- **Circuit Breaker**: A design pattern that prevents repeated calls to a failing service, allowing it to recover
- **Double-Entry Bookkeeping**: Accounting method where every financial transaction creates equal debit and credit entries
- **Idempotency Key**: A unique identifier ensuring that repeating a request produces the same result as the first execution
- **Correlation ID**: A unique identifier passed through all services handling a single request for traceability

Appendix B: Referenced Standards

- **PCI-DSS v4.0** (Payment Card Industry Data Security Standard): Card data handling and storage
- **ISO/IEC 27001** (Information Security Management): Overall security governance
- **RBI Master Direction on PPIs** (Reserve Bank of India — Prepaid Payment Instruments): Wallet limits and KYC norms

- **OWASP Top 10 (2025)** (Open Web Application Security Project): Web application security baseline
- **IEEE 830-1998** (IEEE Recommended Practice for SRS): SRS document structure reference
- **CERT-In Directive (2022)** (Indian Computer Emergency Response Team): Incident reporting timelines

Appendix C: API Endpoint Summary

Method	Endpoint	Description	Auth
POST	**/api/v1/auth/register**	Register new user	Public
POST	**/api/v1/auth/login**	Authenticate user	Public
POST	**/api/v1/auth/verify-otp**	Verify OTP for 2FA	JWT
POST	**/api/v1/auth/refresh**	Refresh access token	Refresh Token
GET	**/api/v1/wallet/balance**	Get wallet balance	JWT + 2FA
POST	**/api/v1/wallet/add-funds**	Add funds to wallet	JWT + 2FA
POST	**/api/v1/payments**	Initiate payment	JWT + 2FA + Idempotency Key
GET	**/api/v1/payments/{txnId}**	Get transaction details	JWT
GET	**/api/v1/payments/history**	List transaction history	JWT
POST	**/api/v1/refunds**	Request refund	JWT + 2FA
GET	**/api/v1/admin/dashboard**	System health dashboard	JWT (Admin)
PUT	**/api/v1/admin/gateways/{id}**	Update gateway config	JWT (Admin)
GET	**/api/v1/admin/reports/audit**	Generate audit report	JWT (Admin)
POST	**/api/v1/admin/kyc/verify**	Verify KYC document	JWT (Admin)
GET	**/api/v1/merchant/settlements**	View settlement history	JWT (Merchant)

Appendix D: Class Diagram — Core Domain Model

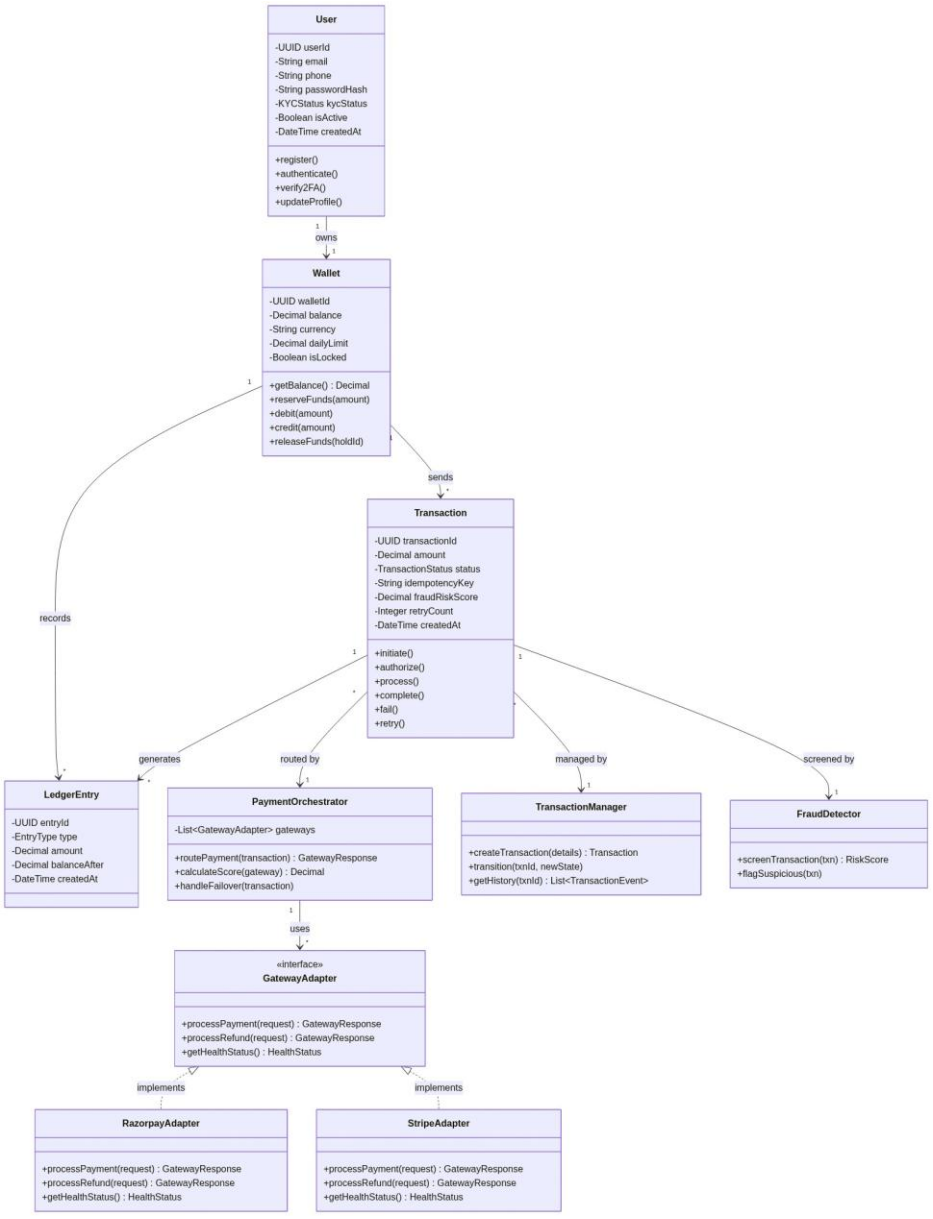


Figure 15.1: Core Domain Object Model & Object-Oriented Class Diagram

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis (30%)	Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis.	Demonstrates good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding; major aspects missing or incorrect.
Functional and Non-Functional Requirement Identification (25%)	Clearly identifies comprehensive, accurate, and well-classified functional and non-functional requirements.	Identifies most requirements with minor classification errors.	Identifies only basic requirements; several omissions.	Requirements are incomplete, inaccurate, or poorly classified.
Actor Identification and Use Case Modelling (20%)	Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards.	Most actors and use cases identified; diagram has minor errors.	Limited actors/use cases identified; diagram partially correct.	Actors/use cases missing or incorrect; diagram absent or inaccurate.
Software Requirement Specification (SRS) Documentation (15%)	SRS is well-structured, complete, professionally organized, and includes all required sections.	SRS is organized with minor missing sections or formatting issues.	SRS includes only essential sections with limited detail.	SRS is incomplete, poorly organized, or lacks essential components.
Assumptions, Constraints, and Requirement Validation (10%)	Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification.	Identifies most assumptions and constraints; validation is adequate.	Limited assumptions/constraints; validation is superficial.	Assumptions, constraints, and validation are missing or incorrect.

Criteria	**Mark**
Scenario Understanding and Problem Analysis (30%)	/5
Functional and Non-Functional Requirement Identification (25%)	/5
Actor Identification and Use Case Modelling (20%)	/5
Software Requirement Specification (SRS) Documentation (15%)	/5
Assumptions, Constraints, and Requirement Validation (10%)	/5
TOTAL	/25